

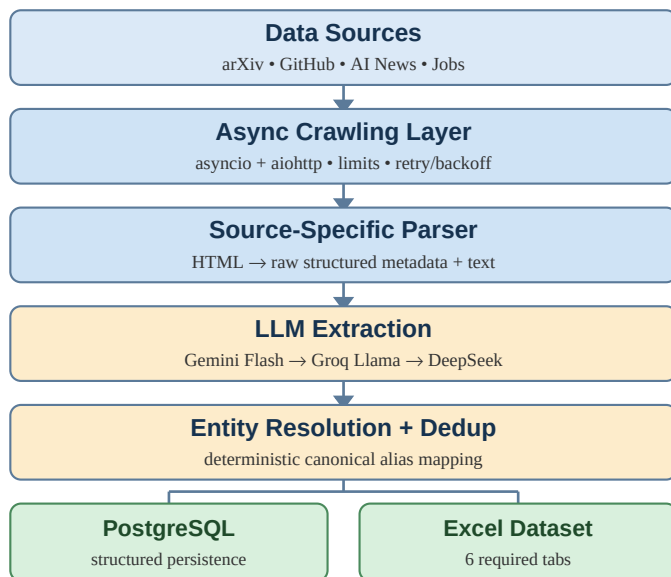
Intelligence Graph Pipeline

System Architecture & Engineering Design — AI
Engineer Demo Task

1. System Overview

Collects structured information from heterogeneous AI-related sources (research papers, GitHub repos, news, job boards) by combining asynchronous crawling, source-specific parsers, LLM-based structured extraction with fallback orchestration, deterministic entity resolution, deduplication, PostgreSQL persistence, Excel dataset generation, and retry/fault-tolerance. The current build focuses on a working research-paper pipeline: async crawling, GitHub correlation, LLM extraction, entity resolution, and database persistence. Additional sources/workers can be added horizontally without changing the core model.

2. High-Level Architecture



3. Implemented Components

3.1 Research Paper Crawler

Crawls arXiv metadata: title, authors, paper URL, publication date, schema version, record type. Output follows a required JSON schema (schemaVersion, recordType=RESEARCH_PAPER, title, authors, paper_url, github_url, github_stars, published_date).

3.2 GitHub Correlation

For each paper, searches GitHub by title via the repo search API plus a title/description matching heuristic, capturing repo name, URL, star count, and description — a traceable link: Research Paper → (title search) → GitHub Repository (URL + stars). Matching is deterministic and transparent; ambiguous matches are never fabricated.

4. Asynchronous Processing

Built on Python asyncio + aiohttp so multiple URLs process concurrently. TCPConnector(limit=N) caps concurrency; asyncio.gather(...) runs many paper tasks in parallel for far better throughput than sequential requests. The same pattern scales horizontally via multiple crawler workers.

5. Reliability & Retry Strategy

Handles network errors, HTTP 429/5xx, connection failures, and slow responses via exponential backoff + jitter (e.g. attempt 1 fails → wait 1s+jitter → attempt 2 fails → wait 2s+jitter → attempt 3 → success/failure). Jitter avoids synchronized retry spikes; timeouts prevent workers blocking forever.

6. LLM Extraction Layer

Fallback chain Gemini Flash → Groq Llama → DeepSeek converts raw content into structured JSON (not free text). The extractor parses fenced JSON, truncates oversized content, retries failures, and preserves source URLs — reducing hallucinated metadata and keeping entities traceable.

7. News Monitoring

Covers AI news sources (TechCrunch AI, The Verge AI) via: collect article candidates → strip navigation links → make URLs absolute → dedupe → fetch pages → extract publication timestamps → extract article text → apply a last-24-hour filter. Tested against real articles.

8. Entity Resolution

Deterministic canonical alias mapping, e.g. “Open AI”/“OpenAI Inc.”/“OpenAI Inc” → **OpenAI**; “DeepMind” → **Google DeepMind**; “Microsoft Corp” → **Microsoft**; “Meta Platforms” → **Meta**. Each resolution logs raw entity, canonical entity, type, and reason — an auditable Entity Mapping Log.

9. Data Storage

PostgreSQL is the structured persistence layer. The research-paper table stores id, schema_version, record_type, title, authors, paper_url (unique), github_url, github_stars, published_date — extensible to startups, products, jobs, news, entities, entity_mappings, crawl_runs.

10. Dataset Generation

Generates an Excel workbook with six required tabs: Startups, Products, Research Papers, Jobs, News, Entity Mapping Log. Research Papers is populated from verified data; other tabs stay empty until real source data exists, per the no-fabrication data-quality rule.

11. Scalability Design

Targets hundreds of thousands of records via: Load Balancer → Worker 1..N → Message Queue → Parser

1..N → LLM Workers → Entity Resolution → PostgreSQL. Scaling adds worker instances rather than changing logic; the queue gives work distribution, backpressure, retry isolation, and failure recovery.

12. Deduplication & Freshness

Stable identifiers (paper_url, article_url, repository URL) plus PostgreSQL uniqueness constraints prevent duplicates. Monitored sources also track first_seen_at, last_seen_at, published_at, and updated_at to distinguish new from previously processed records.

13. JavaScript & Anti-Bot Sites

For sites needing JS rendering, dynamic API calls, or anti-bot handling: try aiohttp first → if successful, process; if JS-heavy/blocked, fall back to Playwright (async) → process. The crawler also respects robots.txt, rate limits, request delays, connection limits, caching, and retry policies.

14. Storage Evolution

Beyond PostgreSQL (structured entities, metadata, source URLs, timestamps, dedup, audit), production storage can add a **Vector Database** (semantic retrieval, similarity search, embeddings) and a **Graph Database** (Company—develops→Product, Company—publishes→Research Paper, Research Paper—implemented_by→GitHub Repo, Company—hiring→Job, News—mentions→Company).

15. Fault Tolerance

Failure isolation: one failed record never halts the run (Record A succeeds, B retries then succeeds, C retries then fails and is logged, D succeeds). Failures capture source URL, error type, timestamp, retry count, and worker info for independent reprocessing.

16. Implementation Status

Implemented & Tested
• Async arXiv crawling & GitHub search/stars • Async GitHub integration • Retry w/ exponential backoff + jitter • PostgreSQL + research-paper persistence • LLM structured extraction (Gemini→Groq→DeepSeek) • Content truncation • Deterministic entity resolution + mapping log • Excel dataset generation • AI news crawling + publication-date extraction • Last-24-hour filtering • Full-text article extraction
Designed for Production Scaling
• Distributed worker architecture & message queue • Horizontal scaling • Playwright fallback • Vector database & graph database • Distributed freshness management • Centralized observability

Production items above are architectural extensions, not deployed claims.

17. Data Quality Principles

1. No fabricated records — only legitimate source-URL data is stored. **2. Source traceability** — every record keeps its source URL. **3. Deterministic normalization** — aliases resolve via explicit mappings, not guesses. **4. Failure over hallucination** — unverifiable/failed extraction is logged, never invented.

18. Conclusion

The pipeline combines asynchronous ingestion, source-specific extraction, LLM-based structuring, deterministic entity resolution, retry handling, and persistent storage into a working end-to-end foundation for research-paper ingestion — demonstrating the architecture needed to expand into news, jobs, startups, products, and further intelligence sources. The design is modular: growth in data volume is handled by adding crawler/parser/LLM worker capacity, not rewriting the pipeline.